

# Note technique

## Projet 8 - IA Engineer

### *Traitez les images pour le système embarqué d'une voiture autonome*

#### Table des matières

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Introduction.....                                           | 1  |
| 2. État de l'art en segmentation sémantique.....               | 2  |
| 2.1. Approches classiques (avant le deep learning) .....       | 2  |
| 2.2. Réseaux convolutionnels .....                             | 3  |
| a) Fully Convolutional Networks (FCN).....                     | 3  |
| b) U-Net .....                                                 | 4  |
| c) Variantes avancées.....                                     | 4  |
| 2.3. Fonctions de perte adaptées.....                          | 5  |
| 3. Présentation du modèle retenu : U-Net .....                 | 7  |
| 3.1 Sans encodeur pré-entraîné.....                            | 7  |
| 3.2 Avec encodeur pré-entraîné VGG16 .....                     | 8  |
| 3.3 Hyperparamètres testés.....                                | 9  |
| 4. Jeu de données et préparation .....                         | 9  |
| 4.1 Structure et contenu du dataset .....                      | 9  |
| 4.2 Prétraitements appliqués.....                              | 10 |
| 4.2 Augmentation de données.....                               | 10 |
| 5. Résultats obtenus.....                                      | 11 |
| 5.1 Courbes d'apprentissage.....                               | 11 |
| 5.2 Comparaison de plusieurs configurations sur 5 epochs ..... | 12 |
| 5.3 Analyse par classe .....                                   | 13 |
| 5.4 Analyse visuelle .....                                     | 14 |
| 5.5 Choix final du modèle .....                                | 15 |
| 6. Conclusion .....                                            | 17 |

## 1. Introduction

L'essor de la vision par ordinateur, couplé aux avancées en intelligence artificielle, a profondément modifié notre capacité à interpréter automatiquement des images. Parmi les tâches les plus complexes et critiques dans ce domaine figure la segmentation sémantique, qui consiste à assigner une étiquette de classe à chaque pixel d'une image. Cette granularité permet une compréhension fine de la scène, essentielle pour des applications comme la conduite autonome, la cartographie intelligente, ou la gestion de l'espace urbain.

Dans un contexte urbain, la segmentation sémantique se heurte à plusieurs défis spécifiques :

- La forte variabilité des scènes : les environnements urbains sont très hétérogènes. D'un quartier à l'autre, les bâtiments changent d'apparence, les routes varient en largeur, les véhicules diffèrent en forme et couleur, et les conditions d'éclairage évoluent au fil de la journée. Un modèle efficace doit donc apprendre à généraliser malgré ces nombreuses variations.
- Le déséquilibre des classes : certaines catégories comme « route », « ciel » ou « bâtiment » occupent une grande partie des images, tandis que d'autres, comme les « piétons » ou les « feux de signalisation », n'apparaissent que de manière ponctuelle. Ce déséquilibre peut amener le modèle à ignorer les classes minoritaires, pourtant souvent cruciales pour les applications pratiques.
- Les limites du nombre d'annotations : produire des masques d'annotation précis au niveau pixel est une tâche extrêmement coûteuse en temps et en ressources humaines. Cela réduit la taille et la diversité des jeux de données disponibles, ce qui complique encore l'apprentissage de modèles profonds sans sur-apprentissage.
- La complexité des objets segmentés : certaines classes présentent des frontières floues ou des formes complexes (ex. : feuillages d'arbres, bordures de trottoir, vélos), ce qui rend difficile leur délimitation précise par le modèle.

Pour répondre à ces défis, nous avons développé un pipeline basé sur l'architecture U-Net, reconnue pour son efficacité dans les tâches de segmentation, même avec des jeux de données limités, combiné à diverses techniques de prétraitement, d'augmentation de données, et à l'utilisation de fonctions de perte adaptées aux données déséquilibrées.

Dans les sections suivantes, nous présenterons dans un premier temps un état de l'art des méthodes existantes en segmentation d'images. Nous détaillerons ensuite l'architecture du modèle retenu, ainsi que les choix techniques qui ont guidé son implémentation. Nous décrirons le jeu de données utilisé, les stratégies d'augmentation appliquées pour améliorer la robustesse du modèle, et enfin, nous analyserons les résultats obtenus. Cette étude se conclura par une discussion sur les limitations observées et les pistes d'amélioration envisageables pour aller au-delà des performances actuelles.

## 2. État de l'art en segmentation sémantique

La segmentation sémantique a connu une évolution rapide au cours des dernières années, portée par les progrès des réseaux de neurones profonds et la disponibilité accrue de données annotées. Cette section présente un panorama des principales approches utilisées pour les scènes urbaines.

### 2.1. Approches classiques (avant le deep learning)

Avant l'avènement du deep learning ou apprentissage profond, la segmentation d'image reposait sur une de ces 3 techniques :

- **Descripteurs manuels** (HOG, SIFT, SURF) pour extraire des caractéristiques locales (orientation des gradients, points clés invariants à l'échelle ou à la rotation) servant ensuite à alimenter des classifieurs (SVM, Random Forest, ou k-NN) pour prédire la classe.
- **Méthodes de regroupement** (k-means, Mean Shift) qui segmente l'image sans supervision pour regrouper les pixels en fonction de similarités de couleur ou de texture.

- **Modèles probabilistes** tels que les champs de Markov ou les champs conditionnels aléatoires (CRF) qui modélise la dépendance entre les pixels pour prendre en compte le fait que les pixels proches sont souvent liés.

Ces méthodes présentaient une faible capacité de généralisation, une faible robustesse aux variations et une précision limitée, notamment dans des environnements complexes.

## 2.2. Réseaux convolutionnels

L'introduction des Convolutional Neural Networks (CNN) a profondément transformé le traitement d'image. Contrairement aux réseaux de neurones traditionnels à couches entièrement connectées (fully connected), les CNN exploitent des couches convolutionnelles qui extraient automatiquement des caractéristiques locales de l'image (textures, contours, formes) tout en réduisant le nombre de paramètres.

Dans le contexte de la segmentation sémantique, les CNN ont été adaptés pour produire une carte de segmentation dense, c'est-à-dire une prédiction de classe pour chaque pixel. Une architecture typique de segmentation par CNN repose sur trois éléments clés :

- **Un encodeur (downsampling)** : il extrait des caractéristiques visuelles de plus en plus abstraites via des convolutions et des opérations de pooling, tout en réduisant la résolution spatiale de l'image.
- **Un décodeur (upsampling)** : il reconstruit la carte de segmentation à la taille originale de l'image grâce à des techniques telles que la transposed convolution (déconvolution) ou l'interpolation.
- **Des connexions par saut (skip connections)** : elles relient directement des couches symétriques du décodeur et de l'encodeur, afin de restaurer les détails spatiaux fins perdus lors du downsampling. Cela améliore significativement la précision des contours et des objets.

Initialement conçus pour la classification, les CNN ont évolué pour la segmentation grâce à des architectures spécifiques, parmi lesquelles :

### a) Fully Convolutional Networks (FCN)

Conçus en 2015, les FCN ont été les premiers à adapter les CNN à la segmentation pixel-par-pixel. L'idée est de conserver une structure entièrement convolutionnelle, capable de produire une sortie de même taille spatiale que l'entrée.

Dans ce modèle, les skip connections sont optionnelles (FCN-32s n'en a pas, FCN-16s en ajoute une, FCN-8s en ajoute deux). Elles sont souvent connectées uniquement à quelques couches intermédiaires du bas du réseau vers la partie haute. Cependant, les bords restent flous, car les informations fines sont partiellement perdues par les couches de pooling.

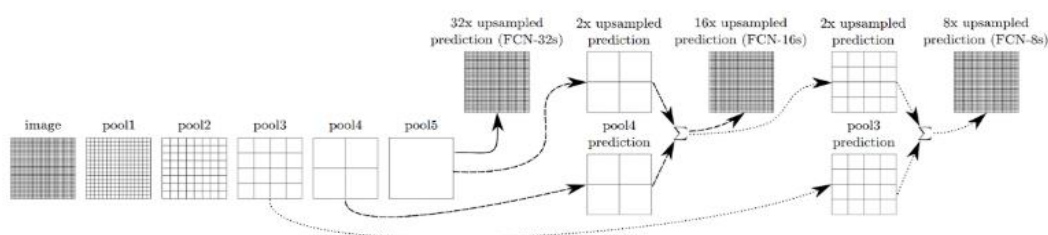


Figure 1: Fully Convolutional Networks for Semantic Segmentation (Shelhamer et al, 2016)

## b) U-Net

Introduite en 2015 pour la segmentation biomédicale, domaine souvent caractérisé par peu de données disponibles et un besoin de grande précision spatiale, U-Net s'est vite imposée comme une architecture de référence, notamment pour les tâches nécessitant une bonne localisation.

Sa spécificité est son architecture symétrique en forme de U. Chaque couche de l'encodeur est connectée directement à sa couche miroir dans le décodeur avec des skip connections systématiques. À chaque niveau, les caractéristiques du chemin descendant sont concaténées (et non ajoutées comme dans le modèle FCN) à celles du chemin ascendant. Cela permet au modèle de restaurer les détails spatiaux fins (bords, textures, petites régions).

U-Net est aujourd'hui largement utilisé pour la segmentation urbaine, notamment en raison de sa capacité à bien gérer les petits objets et les frontières complexes.

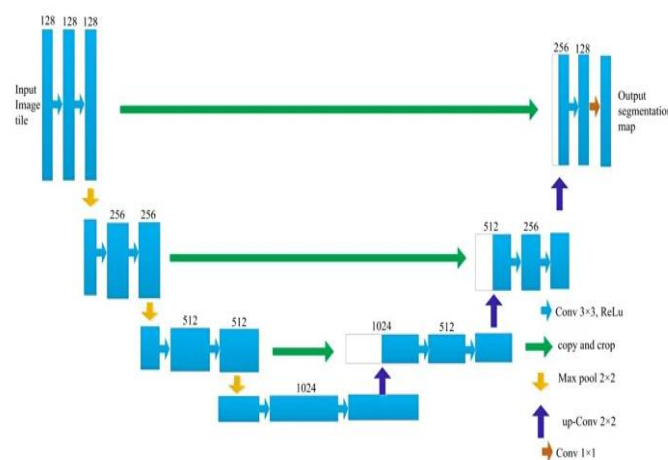


Figure 2: An improved 3D-UNet-based brain hippocampus segmentation model based on MR images (Yana. O et al. 2024)

## c) Variantes avancées

- **DeepLab (v3/v3+)** : La famille DeepLab (par Google) introduit les convolutions à trous (trous/dilated convolutions) qui insère des "trous" (espaces) entre les poids du noyau pour agrandir le champ réceptif sans augmenter le nombre de paramètres ni réduire la résolution. Le module ASPP (Atrous Spatial Pyramid Pooling) permet d'extraire des caractéristiques à plusieurs échelles simultanément de manière efficace, en combinant plusieurs atrous convolutions avec des taux de dilatation différents.

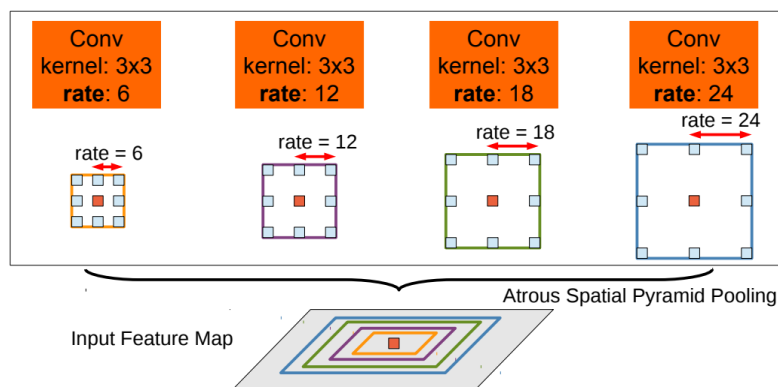


Figure 3: DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs (Liang-Chieh Chen et al, 2016)

- **SegNet** : adopte une approche différente pour le décodeur en mémorisant les indices de pooling (position des maxima) pendant le downsampling pour guider le décodage spatial du upsampling. Cela permet de reconstruire la forme originale des objets sans interpolation.

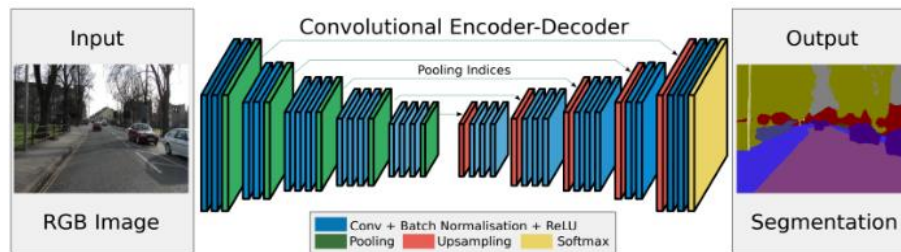


Figure 4: SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation (Vijay Badrinarayanan, 2015)

- **PSPNet** : vise à mieux modéliser le contexte global d'une scène complexe en intégrant un module de pooling pyramidal à plusieurs échelles (global + régional). Cela permet d'agréger les informations globales et locales avant la prédiction finale.

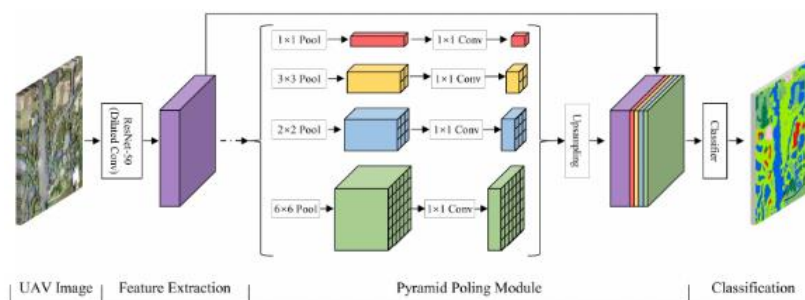


Figure 5: Evaluation of Decision Fusions for Classifying Karst Wetland Vegetation Using One-Class and Multi-Class CNN Models with High-Resolution UAV Images (Yuyang Li, 2022)

## 2.3. Fonctions de perte adaptées

Contrairement à la classification classique (où une seule étiquette est attribuée à une image), la segmentation sémantique produit une étiquette par pixel. La fonction de perte doit donc être conçue pour comparer deux cartes : la carte de prédiction et le masque de vérité terrain.

Un enjeu majeur est la présence de classes déséquilibrées (ex : de petites classes d'objets face à un fond dominant), qui peut biaiser fortement l'apprentissage. Le choix de la fonction de perte a un impact direct sur la capacité du modèle à segmenter correctement les objets rares ou petits.

- **Categorical Cross-Entropy (CCE)** :

C'est la fonction de perte la plus utilisée pour la classification multiclasse. Elle compare les probabilités de prédiction (par pixel) à la classe cible. Toutefois, elle suppose une distribution équilibrée des classes, ce qui n'est souvent pas le cas en segmentation. Elle tend à favoriser les classes majoritaires (comme le fond), et à négliger les objets minoritaires.

- **Dice Loss :**

Basée sur le Dice Coefficient, elle mesure l'intersection (overlap) entre le masque prédit et le masque réel. Plus l'overlap est important (donc meilleure prédiction), plus le Dice coefficient est proche de 1 et la perte est faible.

Elle est particulièrement efficace pour les jeux de données déséquilibrés (ex. : segmentation médicale) car elle se concentre directement sur l'intersection des zones bien prédites et pénalise fortement les erreurs de détection d'objets rares.

- **IoU Loss (Intersection over Union / Jaccard loss) :**

L'Intersection over Union (IoU), ou coefficient de Jaccard, est similaire au Dice, mais elle compare l'intersection (pixels correctement prédits ou vrais positifs VP) à l'union (pixels soit dans la prédiction, soit dans la réalité donc VP + FP + FN).

La différence principale avec la Dice Loss est que l'union est plus sévère (car elle inclut toutes les erreurs, positives ou négatives). Cela rend la IoU Loss plus sensible aux FP et FN, ce qui peut être un avantage en présence de contours bruités ou d'objets mal délimités.

- **Focal Loss :**

Introduite pour gérer les jeux de données avec un fort déséquilibre, elle modifie la Cross-Entropy pour réduire le poids des exemples bien classés et augmenter l'impact des pixels difficiles ou des erreurs sur les classes rares. Elle est souvent utilisée pour améliorer la détection de petits objets (comme les piétons, les panneaux) dans un contexte urbain.

- **Combinaisons de fonctions de perte :**

Pour bénéficier de plus d'avantages, on utilise souvent des fonctions mixtes. Par exemple :

- **Dice + CCE** : on combine la stabilité d'optimisation de la Cross-Entropy avec la sensibilité au recouvrement du Dice.
- **Focal + Dice** : favorise à la fois les pixels mal classés et les objets rares.
- **Tversky Loss** : une généralisation du Dice qui permet d'ajuster le poids des faux positifs vs faux négatifs (utile en médecine).

Quand il y a un déséquilibre de classes dans le jeu de données (ex : beaucoup de route, peu de piétons) : la moyenne simple peut masquer de très mauvais scores sur les classes rares. Dans ces cas là, il vaut mieux utiliser une moyenne pondérée, ou des fonctions de perte par classe pour compenser.

Enfin, le choix de la fonction de perte doit être aligné avec les objectifs de la tâche :

- En cas de classes équilibrées : CCE suffit.
- Pour des classes rares ou des objets fins : privilégier Dice, IoU ou Focal.
- Pour des objets critiques (ex. : sécurité routière, médical), les fonctions combinées sont souvent les plus robustes.

### 3. Présentation du modèle retenu : U-Net

Le modèle choisi pour la segmentation sémantique dans le cadre de notre projet est une architecture de type U-Net, largement reconnue pour ses performances dans les tâches où la précision spatiale est cruciale. Je l'ai implémenté de manière flexible pour permettre l'utilisation soit d'un backbone pré-entraîné (comme VGG16), soit d'un modèle "from scratch" c'est à dire sans poids pré-entraînés. Elle respecte la structure symétrique classique U-Net composée d'un encodeur qui prend en entrée une image de taille 224x224, et d'un décodeur, avec des connexions de type "skip" entre les couches de niveau équivalent, comme vu au chapitre 2 précédent.

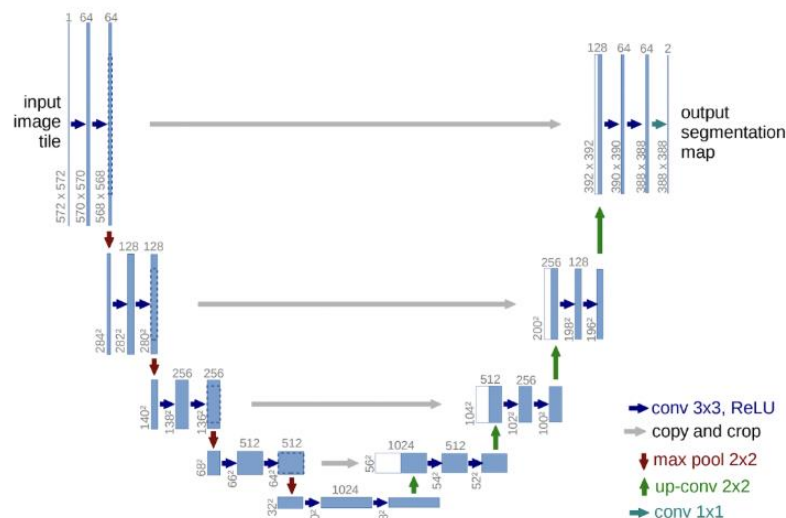


Figure 6: U-Net: Convolutional Networks for Biomedical Image Segmentation (Ronneberger et al, 2015)

#### 3.1 Sans encodeur pré-entraîné

Les modèles entraînés « from scratch » doivent apprendre tout depuis zéro, y compris les couches de bas niveau. Il sont donc souvent utilisés pour des expérimentations légères ou lorsque les données sont très spécifiques (et qu'un backbone généraliste ne serait pas adapté).

L'encodeur permet d'extraire progressivement des caractéristiques de plus haut niveau tout en réduisant la résolution spatiale ( $224 \rightarrow 112 \rightarrow 56 \dots$ ). Ici, j'ai utilisé 3 blocs pour l'encodeur (filter=[64,128,256]) constitué de :

- 2 couches de convolutions Conv2D (3x3) suivies d'une activation ReLU.

La convolution sert à extraire des caractéristiques visuelles de l'image : bords, textures, formes,... Elle applique un filtre  $3 \times 3$  à l'image puis "glisse" sur l'image, pixel par pixel, et à chaque position, il effectue un produit scalaire entre le filtre et la portion de l'image. Le résultat est une nouvelle image qui met en valeur le motif que le filtre détecte.

Après plusieurs niveaux et une augmentation progressive du nombre de filtres, le réseau a appris des détails de plus en plus complexes et le nombre de canaux de caractéristiques augmente à chaque descente (1 pour chaque filtre).

Une fonction d'activation est appliquée après chaque convolution pour introduire de la non-linéarité dans le réseau. ReLU est très simple, si la valeur est  $> 0$ , on la garde, sinon on la remplace par 0.

- 1 MaxPooling2D (2x2) pour réduire la taille spatiale tout en conservant l'information principale, et rendre le réseau plus invariant aux petites translations.

Le max pooling 2x2 prend chaque bloc de 2x2 pixels et en garde le maximum. Cela divise par deux la taille de l'image, tout en gardant l'information la plus forte (la plus activée).

- 1 Dropout (selon le paramétrage) pour éviter le surapprentissage.

À chaque niveau, la sortie avant le pooling est stockée pour les skip connections.

Ensuite le bottleneck correspond à deux convolutions 3x3 avec un nombre de filtres doublé par rapport au dernier niveau de l'encodeur. C'est la partie la plus profonde du réseau : elle a une vision très globale de l'image (petite dimension spatiale, grande profondeur). C'est ici que les caractéristiques les plus abstraites, complexes sont apprises.

Enfin, le décodeur réalise une expansion progressive des cartes de caractéristiques (feature maps) pour revenir à la taille d'origine de l'image. Pour cela, le nombre de filtres diminuent au fur et à mesure de la remontée (ex :  $512 \rightarrow 256 \rightarrow 128 \rightarrow \dots$ ). À chaque étape :

- 1 UpSampling2D (2x2) qui double la résolution spatiale.
- Les skip connections permettent la concaténation avec les features correspondantes symétriques de l'encodeur pour récupérer les informations "perdues" via le maxpooling de l'encodeur et de les réinjecter lors de la remontée du décodeur.
- 2 couches de convolutions Conv2D (3x3) suivies d'une activation ReLU pour réaffiner les détails.

La sortie finale passe par une convolution  $1 \times 1$  pour réduire le nombre de canaux de sortie. Une activation softmax transforme pour chaque pixel les vecteurs de scores de tous les canaux en probabilités afin de produire une carte de probabilité par pixel pour chaque classe, ici 8.

### 3.2 Avec encodeur pré-entraîné VGG16

Le backbone VGG16 est un réseau déjà entraîné sur la grande base d'images ImageNet qui a donc appris à extraire des caractéristiques visuelles très efficaces (bords, textures, formes...) dès les premières couches. On réutilise les poids de sortie de VGG16 en les gelant pour accélérer l'apprentissage et améliorer la qualité des features, notamment sur des jeux de données restreints, cela s'appelle le transfert d'apprentissage.

La profondeur du réseau (et donc la **taille des features** dans le bottleneck) est bien plus grande avec VGG16. Cela permet d'extraire des représentations plus abstraites (par exemple, pour détecter la présence d'un objet même s'il est partiellement caché), mais ça coûte aussi plus cher en mémoire et en calcul. Ainsi, il contient 5 blocs de convolutions (3x3) + pooling, qui réduisent progressivement la taille de l'image. Le bottleneck est intégré dans le backbone pré-entraîné.

Le décodeur reconstruit progressivement l'image, comme dans le cas sans encodeur pré-entraîné, via une succession de Upsampling (2x2), skip connections concaténatives, convolutions  $3 \times 3$  avec ReLU, et de Dropout (selon le paramétrage). Comme les poids de l'encodeur sont gelés avec



l'encodeur pré-entraîné, le Dropout a été déplacé dans le décodeur pour régulariser la partie entraînée du réseau. La couche finale applique une convolution  $1 \times 1$  pour produire une carte de sortie avec 8 canaux, activée par la fonction softmax.

L'avantage d'un encodeur VGG16 pré-entraîné est qu'il fournit des représentations robustes et bien structurées dès les premières couches, accélérant la convergence et améliorant la généralisation.

### 3.3 Hyperparamètres testés

Différents paramètres d'apprentissage du modèle ont été testés :

- **Nombre de filtres** : [64, 128, 256] dans l'encodeur, avec symétrie inversée dans le décodeur. Utilisé uniquement pour l'entraînement sans encodeur pré-entraîné.
- **Dropout** : Plusieurs taux ont été testés : 0, 0.3 et 0.5.
- **Optimiseur** : Adam et SGD.
- **Learning rate** : Deux valeurs ont été testées :  $1e-4$  et  $1e-5$ .
- **Loss** : Deux fonctions de perte ont été évaluées : Categorical Cross-Entropy (CCE) et la combinaison Dice + Focal Loss.

## 4. Jeu de données et préparation

Le jeu de données utilisé pour ce projet est Cityscapes, une référence en segmentation sémantique urbaine. Il se compose de 5 000 images RGB haute résolution ( $2048 \times 1024$ ) prises dans 50 villes allemandes et annotées pixel par pixel. Chaque pixel est associé à une classe sémantique représentant un élément de la scène (route, trottoir, voiture, piéton, etc.). Dans notre cas, nous avons considéré 8 classes principales, après fusion ou regroupement de certaines classes rares.

### 4.1 Structure et contenu du dataset

Cityscapes est divisé en deux répertoires principaux :

- **leftImg8bit/** : contient les images RGB d'entrée.
- **gtFine/** : contient les masques d'annotations, incluant :
  - **\_gtFine\_labelIds.png** : identifiants de classes (0 à 33).
  - **\_gtFine\_color.png** : masques colorisés.
  - **\_gtFine\_instanceIds.png** : identifiants d'objets individuels.

Les 5000 images sont réparties comme suit :

- **Train** : 2975 images
- **Validation** : 500 images
- **Test** : 1525 images

## 4.2 Prétraitements appliqués

Avant d'entraîner le modèle, plusieurs transformations ont été appliquées :

- **Regroupement** : les 33 classes originales ont été regroupées en 8 super-catégories («vide», «route/trottoir», «construction», «objet», «nature», «ciel», «humain» et «véhicule»). Cela permet de réduire la complexité tout en conservant l'essentiel de la sémantique urbaine.
- **Redimensionnement** : les images ont été ramenés à une résolution uniforme de 224x224 pixels, ce qui réduit la charge computationnelle tout en conservant les détails essentiels.
- **Normalisation** :
  - **Sans backbone pré-entraîné** : les valeurs RGB sont mises à l'échelle dans  $[0, 1]$ .
  - **Avec VGG16** : on utilise la fonction `preprocess_input` spécifique, qui soustrait la moyenne RGB d'ImageNet pour normaliser dans le style du pré-entraînement (dans  $[0, 255]$  puis centrée autour de 0).
- **Encodage one-hot** : chaque pixel des masques, étiqueté avec un entier correspondant à sa classe, est converti en un vecteur binaire de taille 8 où un seul bit est à 1 (la position correspondant à la classe). Cela permet d'utiliser des fonctions de coût multiclasse comme `categorical_crossentropy`, `dice` ou `focal`.

## 4.2 Augmentation de données

Le dataset présente un déséquilibre significatif entre les classes (par exemple, dominance de la route, du ciel ou des bâtiments vs. rareté des piétons, panneaux ou poteaux).

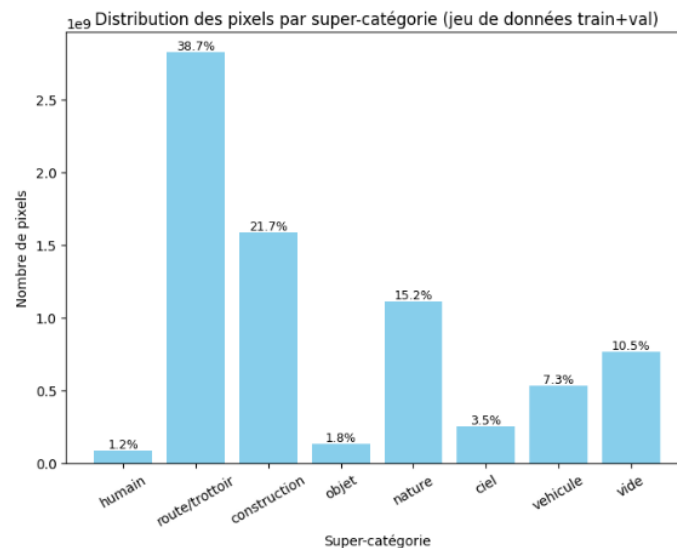


Figure 7: Distribution des pixels des jeu de données train et validation après regroupement en super-catégories

Pour y remédier, la fonction de perte combinée Dice + Focal a été choisie pour permettre de gérer ce déséquilibre naturellement, en accentuant l'impact des erreurs sur les classes rares (via Focal) et en favorisant le recouvrement précis sur les objets fins et peu fréquents (via Dice).

Cependant, pour limiter le surapprentissage et améliorer la robustesse du modèle, des techniques d'augmentation de données ont été intégrées au pipeline :

- **Transformations géométriques** : retournements horizontaux et légères rotations avec décalages aléatoires pour simuler d'autres points de vue.
- **Perturbations photométriques** : légères variations de luminosité et contraste permettent de simuler différentes conditions météorologiques ou d'éclairage.

L'objectif de ces techniques est d'enrichir la diversité des données, en particulier pour les classes rares, et de renforcer la généralisation du modèle sur des situations urbaines variées.

## 5. Résultats obtenus

### 5.1 Courbes d'apprentissage

Deux configurations principales ont été testées :

- Sans backbone (encodeur simple entraîné from scratch).
- Avec VGG16 (backbone pré-entraîné sur ImageNet).

Au vu des temps d'entraînement très longs, les entraînements ont été suivis pendant 5 epochs, puis, un entraînement complet sur 50 epochs avec le meilleur modèle a été réalisé.

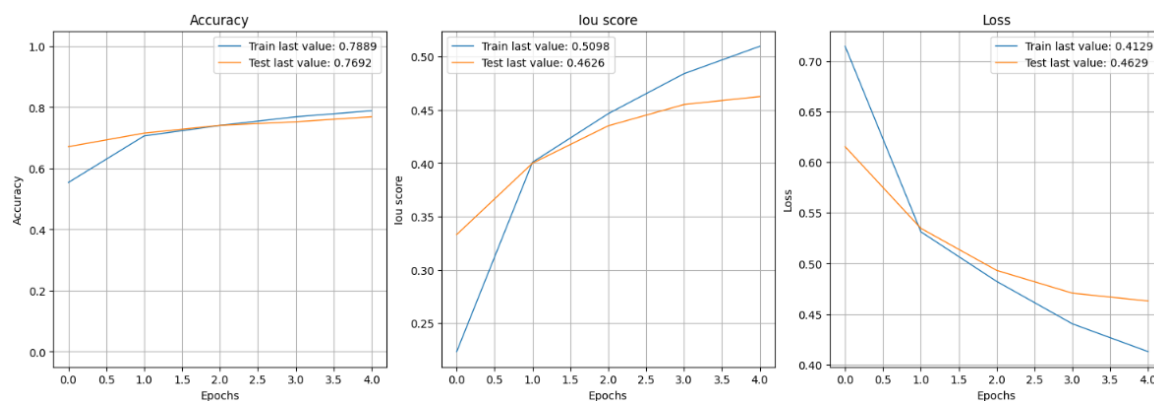


Figure 8: Courbes d'entraînement sans backbone et sans data augmentation : accuracy, IoU et perte

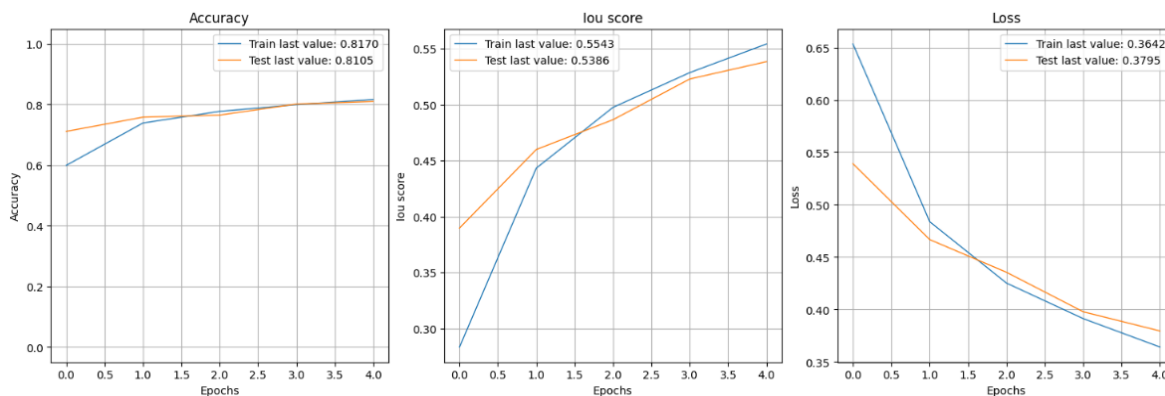


Figure 9: Courbes d'entraînement sans backbone et avec data augmentation : accuracy, IoU et perte

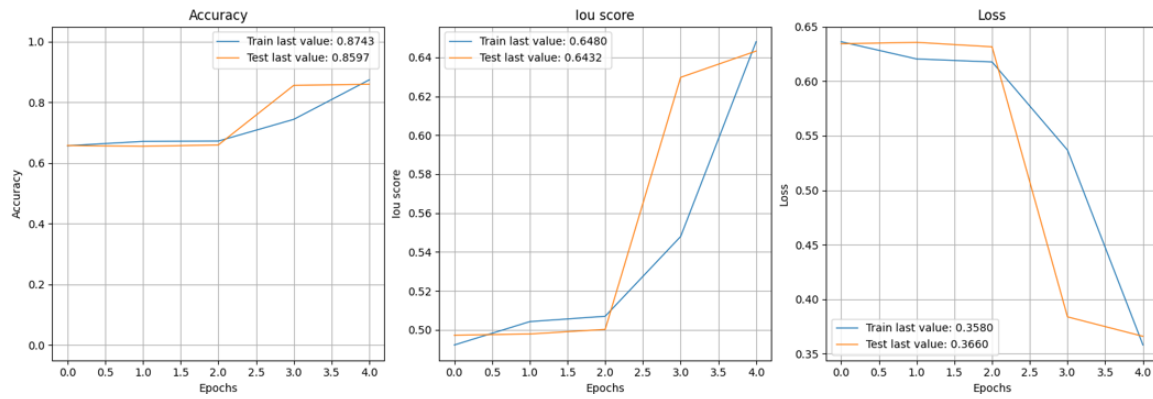


Figure 10: Courbes d'entraînement avec backbone VGG16 et avec data augmentation : accuracy, IoU et perte

On observe que l'ajout de data augmentation améliore la robustesse du modèle sans backbone, réduisant l'écart entre train et validation et augmentant le score IoU. L'introduction du backbone VGG16 permet au modèle de converger plus vite et d'extraire des caractéristiques plus discriminantes dès le début d'où l'augmentation des performances.

## 5.2 Comparaison de plusieurs configurations sur 5 epochs

| Backbone     | Data augmentation | Fonction de perte        | Dropout    | Learning rate            | IoU global   | IoU pondéré  | IoU macro    | Dice global  | Dice pondéré | Dice macro   |
|--------------|-------------------|--------------------------|------------|--------------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Aucun        | Non               | Categorical CrossEntropy | 0.3        | $1e-4$                   | 44.2%        | 64.8%        | 45.5%        | 78.9%        | 76.8%        | 56.1%        |
| Aucun        | Non               | Categorical CrossEntropy | 0.5        | $1e-4$                   | 46.1%        | 66.4%        | 47.3%        | 79.5%        | 77.9%        | 57.4%        |
| Aucun        | Non               | Dice + Focal             | 0.3        | $1e-4$                   | 46.3%        | 63.3%        | 47.6%        | 76.9%        | 75.9%        | 60.2%        |
| Aucun        | Non               | Dice + Focal             | 0.5        | $1e-4$                   | 46.9%        | 63.5%        | 47.9%        | 76.9%        | 76.91        | 60.8%        |
| Aucun        | Oui               | Dice + Focal             | 0.3        | $1e-4$                   | 37.0%        | 53.6%        | 54.9%        | 69.1%        | 67.7%        | 67.3%        |
| Aucun        | Oui               | Dice + Focal             | 0.5        | $1e-4$                   | 52.4%        | 67.5%        | 53.4%        | 78.9%        | 79.4%        | 65.9%        |
| Aucun        | Oui               | Dice + Focal             | 0.3        | $1e-5$                   | 53.8%        | 69.7%        | 37.9%        | 81.0%        | 81.0%        | 49.9%        |
| VGG16        | Non               | Dice + Focal             | 0.3        | $1e-4$                   | 4.8%         | 14.5%        | 4.8%         | 38.1%        | 21.0%        | 6.9%         |
| VGG16        | Non               | Dice + Focal             | 0.5        | $1e-4$                   | 53.2%        | 74.2%        | 54.8%        | 85.1%        | 83.5%        | 65.3%        |
| VGG16        | Non               | Dice + Focal             | 0.3        | $1e-5$                   | 60.8%        | 76.6%        | 63.2%        | 86.1%        | 85.9%        | 74.7%        |
| VGG16        | Non               | Dice + Focal             | 0.5        | $1e-5$                   | 59.1%        | 75.6%        | 61.5%        | 85.6%        | 85.2%        | 73.2%        |
| <b>VGG16</b> | <b>Oui</b>        | <b>Dice + Focal</b>      | <b>0.3</b> | <b><math>1e-4</math></b> | <b>64.6%</b> | <b>79.1%</b> | <b>66.7%</b> | <b>88.0%</b> | <b>87.6%</b> | <b>77.6%</b> |
| VGG16        | Oui               | Dice + Focal             | 0.5        | $1e-4$                   | 34.6%        | 56.9%        | 34.5%        | 71.7%        | 64.7%        | 40.3%        |
| VGG16        | Oui               | Dice + Focal             | 0.3        | $1e-5$                   | 61.3%        | 77.3%        | 63.5%        | 86.7%        | 86.3%        | 74.8%        |
| VGG16        | Oui               | Dice + Focal             | 0.5        | $1e-5$                   | 61.0%        | 76.7%        | 63.2%        | 86.2%        | 86.0%        | 74.7%        |

Les quatre premières lignes montrent que remplacer la fonction de perte Categorical CrossEntropy par Dice + Focal améliore nettement les prédictions puisque les métriques macro (même poids pour chaque classe) augmentent, c'est à dire qu'il y a un meilleur apprentissage des classes minoritaires. Cela confirme que Dice + Focal est mieux adapté aux données déséquilibrées comme celles de Cityscapes, en renforçant l'apprentissage des petites classes.

La data augmentation seule semble peu bénéfique, voire nuisible selon le backbone ou les hyperamètres. Cependant, avec la bonne combinaison, les augmentations améliorent la robustesse.

Avec VGG16, on observe une forte amélioration générale, notamment sur les métriques macro ce qui s'explique par la capacité de VGG16 à capturer des caractéristiques visuelles plus riches, grâce à ses couches pré-entraînées sur ImageNet.

### 5.3 Analyse par classe

Les scores moyens ne suffisent pas à évaluer les performances du modèle sur les **classes rares** (piétons, objets) : une analyse par classe est donc indispensable.

Dans notre cas, le modèle ayant les meilleurs métriques (voir tableau au-dessus), a également les meilleurs métriques par classe :

| Classe          | vide | route/trottoir | construction/bâtiment | objet | nature | ciel | humain | véhicule |
|-----------------|------|----------------|-----------------------|-------|--------|------|--------|----------|
| <b>IoU (%)</b>  | 66.0 | 91.2           | 73.0                  | 21.9  | 79.8   | 86.1 | 41.7   | 73.8     |
| <b>Dice (%)</b> | 79.6 | 95.4           | 84.4                  | 36.0  | 88.8   | 92.6 | 58.8   | 85.0     |

Les résultats sont très bons sur les grandes structures (route, bâtiment, ciel, véhicule), mais restent plus faibles pour les **objets fins et mobiles** (humain, objet) malgré l'utilisation de Dice+Focal Loss.

Observons les erreurs les plus fréquentes grâce à la matrice de confusion :

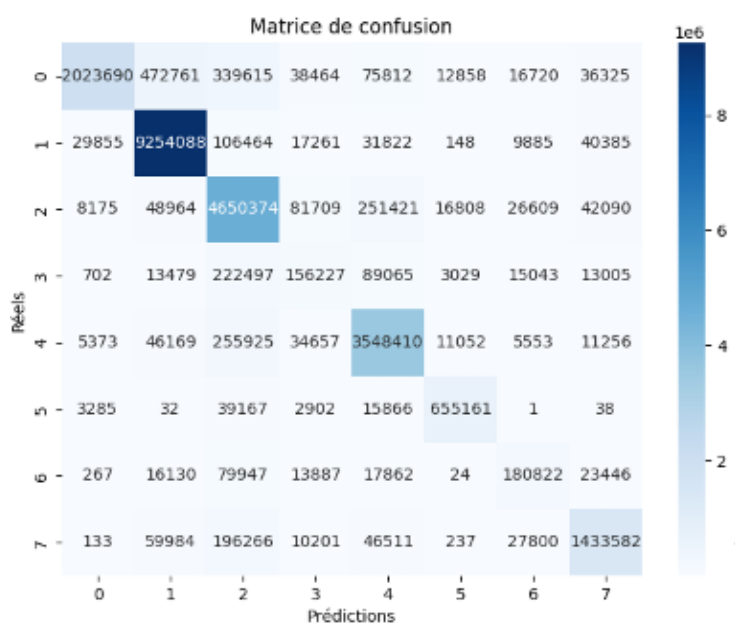


Figure 11: Matrice de confusion du modèle avec encodeur VGG16, data augmentation, dropout 0.3 et learning rate 0.0001

On observe par exemple que la classe "objet" est fréquemment confondue avec "construction/bâtiment".

## 5.4 Analyse visuelle

Enfin, une évaluation visuelle des prédictions a été réalisée en comparant l'image d'entrée, le masque de réel et le masque prédit par le modèle :

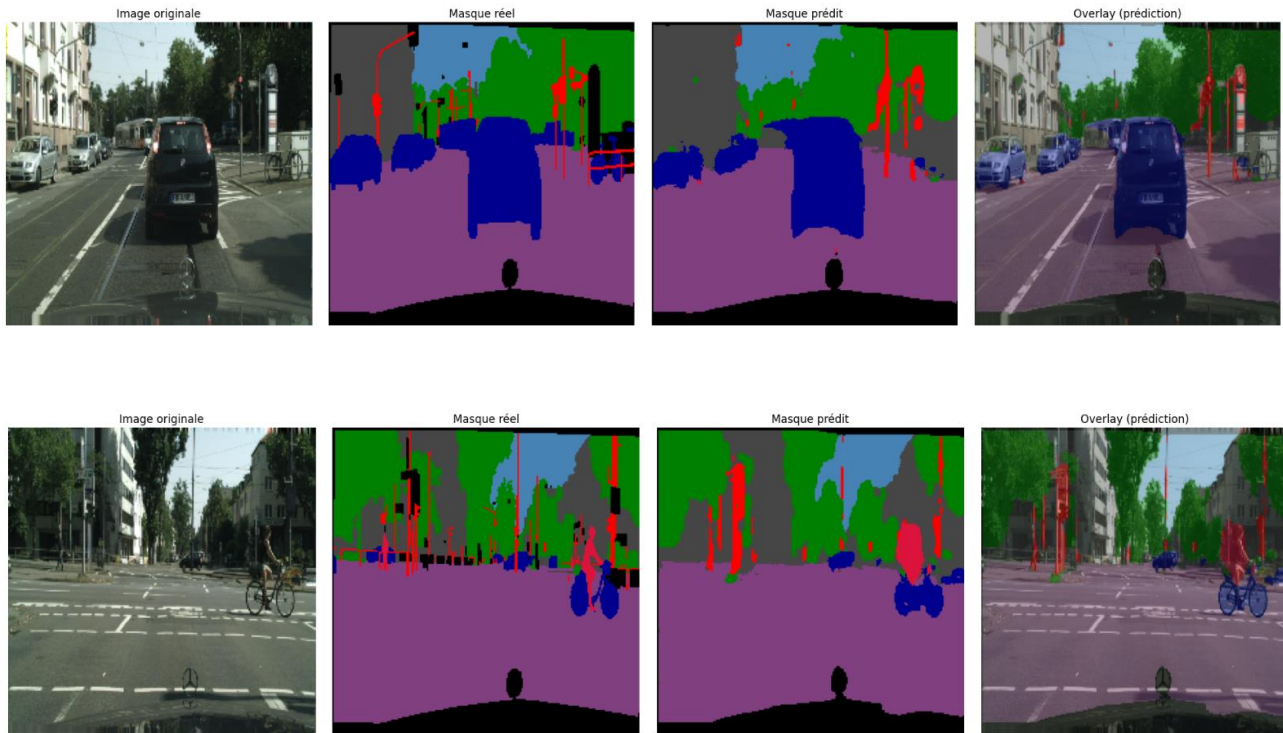


Figure 12: Visualisation des masque prédits avec le modèle avec encodeur VGG16, data augmentation, dropout 0.3 et learning rate 0.0001

Ces visualisations permettent de retrouver les bonnes performances du modèle sur les grandes structure, ainsi que ses difficultés à détecter les objets ou humains. Les objets se fondent souvent avec les bâtiments en arrière-plan.

## 5.5 Choix final du modèle

Après comparaison, le modèle avec :

- encodeur VGG16
- fonction de perte Dice+Focal
- dropout de 0.3
- learning rate de  $1e-4$
- data augmentation

est retenu comme meilleure configuration et a été entraîné sur 50 epochs.

Ce modèle combine la robustesse du pré-entraînement (VGG16) à une régularisation efficace (Dropout + augmentation) et une fonction de perte adaptée au déséquilibre des classes (Dice+Focal).

Cette configuration a permis d’obtenir des performances satisfaisantes sur toutes les classes, en particulier les plus petites, tout en assurant une stabilité d’apprentissage et une bonne convergence.

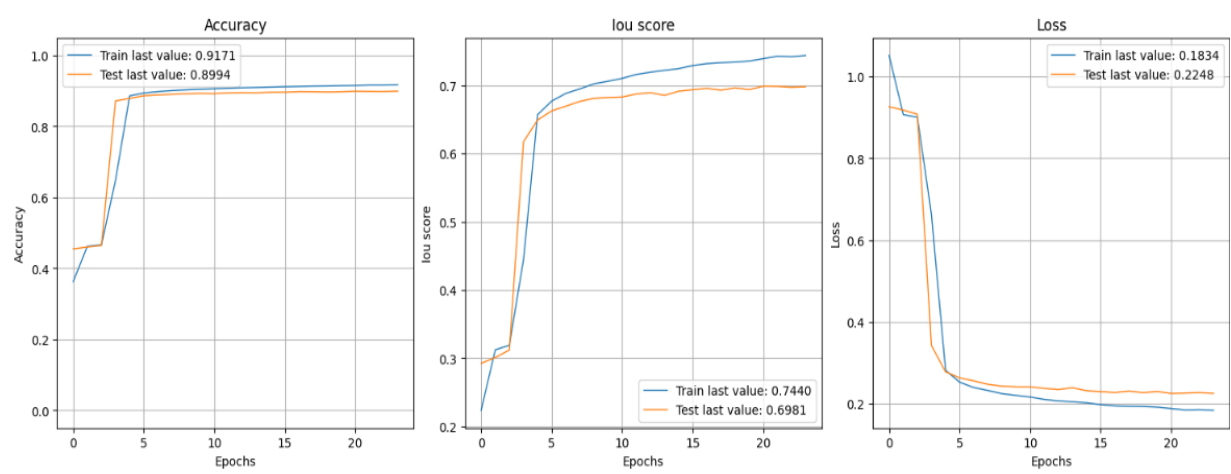


Figure 13: Courbes d'entraînement sur 50 epochs avec modèle final : accuracy, IoU et perte  
Après l’entraînement complet sur 50 epochs, voici les résultats :

On remarque une stabilisation progressive indiquant une convergence du modèle et ici, la courbe de validation ne se dégrade pas significativement après un pic, ce qui suggère que la data augmentation et le dropout ont bien régulé le surapprentissage La courbe de perte de validation est légèrement plus haute que celle du train, mais l’écart reste stable, donc la généralisation est satisfaisante.

| Backbone | Data augmentation | Fonction de perte | Dropout | Learning rate | IoU global | IoU pondéré | IoU macro | Dice global | Dice pondéré | Dice macro |
|----------|-------------------|-------------------|---------|---------------|------------|-------------|-----------|-------------|--------------|------------|
| VGG16    | Oui               | Dice + Focal      | 0.3     | 1e-4          | 69.9%      | 82.1%       | 71.8%     | 88.9%       | 89.6%        | 82.0%      |

| Classe   | vide | route/trottoir | construction/bâtiment | objet | nature | ciel | humain | véhicule |
|----------|------|----------------|-----------------------|-------|--------|------|--------|----------|
| IoU (%)  | 67.0 | 92.1           | 77.8                  | 33.0  | 83.4   | 88.1 | 52.4   | 80.7     |
| Dice (%) | 80.2 | 95.9           | 87.5                  | 49.6  | 91.0   | 93.7 | 68.8   | 89.3     |

Les métriques nous indiquent une très bonnes performances globales et sur les classes fréquentes (ciel, route, véhicule, nature). Elles sont moins bonnes sur les classes rares (objet, humain) mais sont tout de même améliorées avec l’entraînement complet.



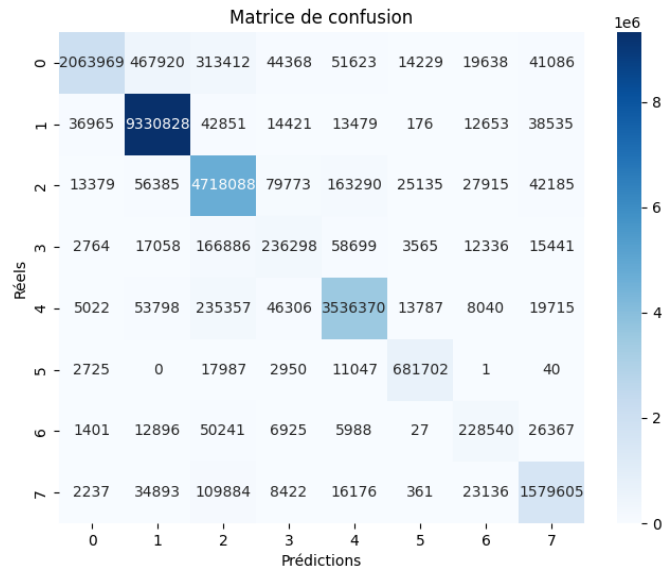


Figure 14: Matrice de confusion du modèle final sur 50 epochs

Sur la matrice de confusion, la diagonale montre une grande majorité de pixels bien classés. On retrouve les confusions entre la classe "objet" et "construction/bâtiment" notées précédemment. Les classes majoritaires sont bien gérées, tandis que les classes rares ou visuellement ambiguës sont parfois mal classées.

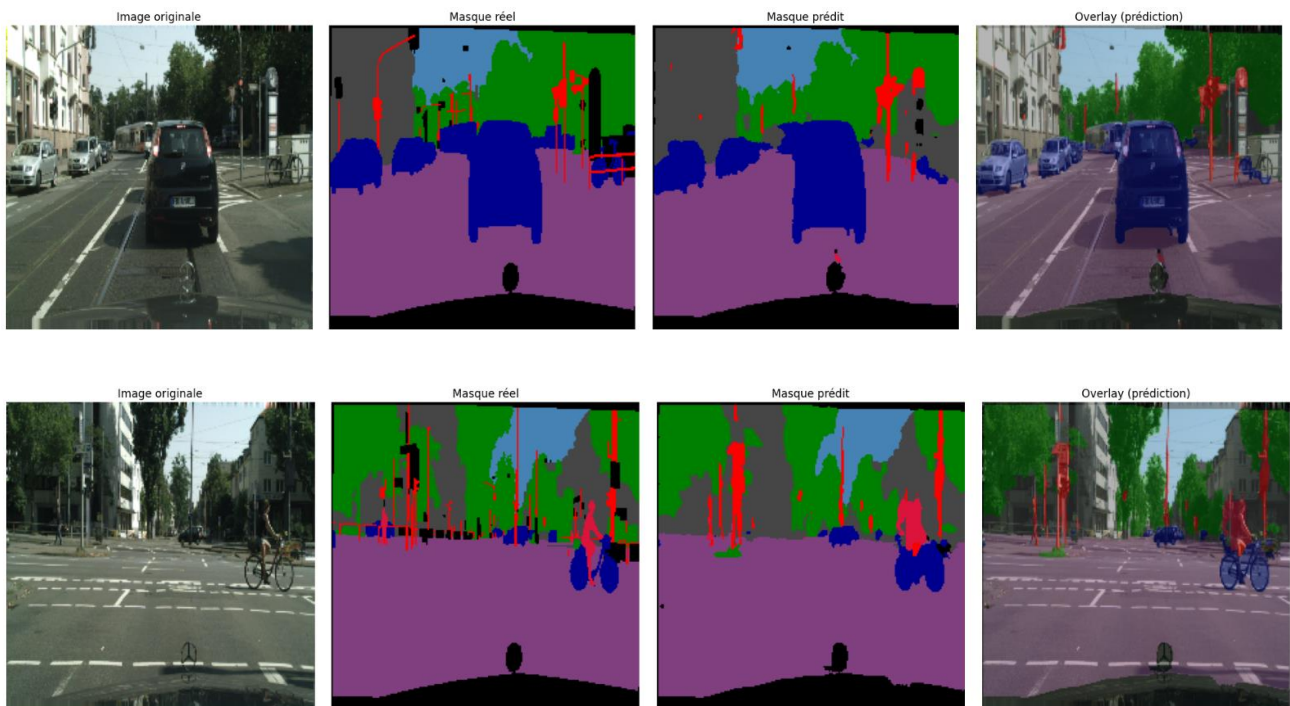


Figure 15: Visualisation des masques prédits avec le modèle final sur 50 epochs

Visuellement, on note une excellente segmentation des grandes structures (route/trottoir, ciel, véhicules ou bâtiments/construction) sont bien capturées. Cela se traduit par une forte précision des contours, peu d'erreurs de classe, et une bonne correspondance visuelle avec le masque réel.

Cependant, les objets comme les panneaux, lampadaires ou poubelles sont souvent fusionnés avec l'arrière-plan ou mal détectés. Ce phénomène est dû à leur taille réduite et leur forme variable. De même, les humains sont parfois incomplets, flous, ou totalement absents des masques prédits.



## 6. Conclusion

Le projet a permis de construire un modèle de segmentation sémantique performant pour des scènes urbaines, avec un pipeline U-Net optimisé. Les principaux enseignements sont :

- Le backbone pré-entraîné est un levier majeur de performance, notamment sur les classes complexes ou les jeux de données limités.
- La fonction de perte Dice + Focal améliore sensiblement la segmentation des petites classes souvent ignorées.
- La data augmentation contribue à la robustesse, même si son effet dépend des hyperparamètres.

Il reste cependant des limitations. Ainsi, la classe "objet" reste mal segmentée, souvent confondue avec "construction / bâtiment". L'apprentissage reste sensible aux déséquilibres de classes extrêmes. La résolution de sortie (224x224) peut limiter la détection de très petits éléments et contribuer à ces limitations, tout comme la taille limitée du dataset.

Pour améliorer les résultats, de nouveaux modèles pourraient être testés, comme DeepLabV3+ pour une meilleure gestion des objets fins grâce au module ASPP ou des Vision Transformers (ViT) comprenant des mécanismes d'attention. En parallèle, le dataset pourrait être étoffé en terme de nombre d'images mais également pour avoir plus de représentations des classes minoritaires.